Real-Time CPA at Scale

Scoping, Node Splitting, and GDS

Pierre Halftermeyer · Field Engineer, Neo4j

2025

Abstract. Critical Path Analysis (CPA) on process DAGs is a standard O(V+E) problem — but on a large, dynamic production graph, the naive approach fails at two distinct points: projecting the full graph is too slow, and enumerating ancestors via Cypher variable-length paths is exponential. This article presents a three-step technique — scoped ancestor collection via `apoc.path.subgraphNodes`, virtual node splitting at projection time, and targeted GDS projection — that brings CPA from hours to milliseconds per scoping node. All queries are production-ready Cypher, demonstrated on a toy dataset with full expected results.

§1. Introduction

Process orchestration systems — batch pipelines, financial clearing chains, manufacturing workflows — are naturally modelled as Directed Acyclic Graphs (DAGs). Tasks depend on other tasks; some can run in parallel, others must wait. At any point in the day, some tasks are already completed and some are still pending. The question that matters operationally is: **given the current state of execution, what is the longest path still remaining?** This is Critical Path Analysis (CPA), and the answer directly drives SLA prediction and incident response.

Neo4j's Graph Data Science library provides `gds.dag.longestPath`, an efficient algorithm for exactly this problem — it runs in $O(V + E)$ using dynamic programming on the topological order of the DAG. A primer on DAGs and this algorithm is available in Unlocking DAGs in Neo4j: From Basics to Critical Path Analysis. This article focuses on a different challenge: making CPA work in production, on a graph that is both **large** and **dynamic**.

Large means tens or hundreds of millions of edges. Dynamic means the graph changes continuously throughout the day — tasks complete, new instances are created, statuses update. These two properties together rule out the most natural approaches, each for a different reason.

§2. Two Distinct Problems

For a given **scoping node** (a top-level task or service entry point), the CPA is the longest remaining path across all pending upstream tasks. There may be multiple scoping nodes in a live graph; the CPA is computed independently for each, on demand.

2.1 Trap 1 — You cannot project the full graph at query time

GDS graphs live in memory. Projecting the full graph is $O(m)$ in the number of edges — linear, not exponential. On a small graph, that's fine.

On a 100M-edge production graph, that projection takes tens of seconds.

The critical point is that **you cannot cache it**. The graph is a living object: task completions arrive continuously throughout the day, and the frontier between done and not-done shifts with every update. A projection taken at 9:00 is stale by 9:01. You must project fresh at query time — and $O(m)$ at query time means no sub-second response.

> **The fix**
>
> Scope the projection to T_{sub} — the subgraph of ancestors of the scoping node. In a large process graph, any individual scoping node's ancestor set is typically orders of magnitude smaller than the full graph. Projecting T_{sub} instead of the full graph brings the cost down to something compatible with real-time querying.

2.2 Trap 2 — You cannot enumerate ancestors with Cypher variable-length paths

The natural way to collect ancestor nodes is a variable-length Cypher pattern: `(scopingNode)←[:PRECEDES]-*(n)`. But Cypher has two match modes, and neither gives you what you need here:

- **`DIFFERENT RELATIONSHIPS`** (Cypher default) — a relationship can only be traversed once *within a given path match*, but the same node can be reached via multiple distinct paths, each returned as a separate result row.
- **`REPEATABLE ELEMENTS`** (GQL default) — relaxes even that constraint, allowing relationships to be traversed multiple times within a single path.

The critical point is that **both modes are path-level constraints** — they govern what is allowed within a single path, independently for each match. Neither mode has any awareness of the set of paths returned as a whole. There is no mechanism to say “if this node was already returned via another path, skip it”.

On a diamond, T_D is reached via T_B→T_D and via T_C→T_D — two distinct paths, two result rows, two independent matches. Each is valid under either mode. The result set accumulates all of them.

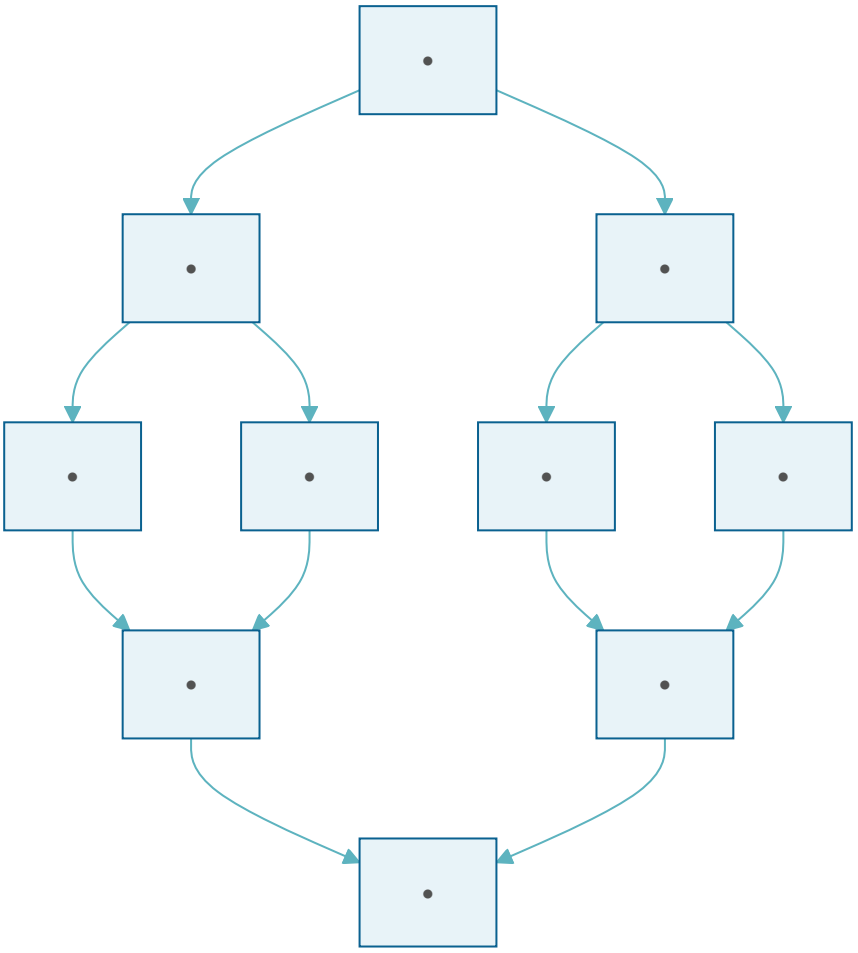

Figure 1 — Multi-level diamond: m parallel branches at each level, d levels deep → m^d paths.

With m parallel branches at each level and d levels, there are m^d **paths** to enumerate. This is exponential in depth. In the production environment this technique was built for, the naive Cypher traversal ran for several hours on a test graph — and was simply not executable in production.

Depth is not the issue. Fan-out is.

The fix

Use **`NODE_GLOBAL` uniqueness** — once a node has been visited, it is never revisited, regardless of which branch led there. This collapses the traversal from exponential in paths to linear in nodes and edges: $O(V_{\text{sub}} + E_{\text{sub}})$. It is the classic BFS/DFS guarantee, and it is what `apoc.path.subgraphNodes` provides. This makes `apoc.path` the right tool here — it provides `NODE_GLOBAL` traversal directly in Cypher, without requiring a separate GDS projection step just to collect the ancestor set.

§3. The Solution in Three Steps

With both traps identified, the solution follows directly. We need to: collect ancestors efficiently (Step 1), encode node durations as edge weights so GDS can process them (Step 2), and project only the relevant subgraph into GDS (Step 3). Steps 1 and 3 are combined in a single query.

3.0 Toy Dataset

Two chained diamonds, a sibling, a cousin branch, and an unrelated branch. The diamonds illustrate fan-out (Trap 2). The sibling, cousin, and unrelated branch illustrate the three categories of nodes that scoping cuts away — including nodes within the same WCC as the scoping node (Trap 1).

```
// ── Create toy graph ──────────────────────────────────────
CREATE
  // Main process chain
  (tA:Task:Done {uid: 1,  name: "T_A", duration: 2.0}),
  (tB:Task      {uid: 2,  name: "T_B", duration: 1.0}),
  (tC:Task      {uid: 3,  name: "T_C", duration: 3.0}),  // ← fi
  (tD:Task      {uid: 4,  name: "T_D", duration: 1.0}),
  (tE:Task      {uid: 5,  name: "T_E", duration: 0.5}),
  (tF:Task      {uid: 6,  name: "T_F", duration: 2.0}),  // ← sec
  (tG:Task      {uid: 7,  name: "T_G", duration: 0.5}),  // ← sc

  // Diamond 1: T_A fans out to T_B and T_C, converges on T_D
```

```
(tA)-[:PRECEDES]→(tB),
(tA)-[:PRECEDES]→(tC),
(tB)-[:PRECEDES]→(tD),
(tC)-[:PRECEDES]→(tD),

// Diamond 2: T_D fans out to T_E and T_F, converges on T_G
(tD)-[:PRECEDES]→(tE),
(tD)-[:PRECEDES]→(tF),
(tE)-[:PRECEDES]→(tG),
(tF)-[:PRECEDES]→(tG),

// Sibling: T_F also precedes T_H
(tH:Task {uid: 8,  name: "T_H", duration: 1.5}),
(tF)-[:PRECEDES]→(tH),

// Cousin branch: T_D → T_I → T_J
(tI:Task {uid: 9,  name: "T_I", duration: 1.0}),
(tJ:Task {uid: 10, name: "T_J", duration: 0.5}),
(tD)-[:PRECEDES]→(tI),
(tI)-[:PRECEDES]→(tJ),

// Unrelated branch — separate WCC
(tX:Task {uid: 11, name: "T_X", duration: 1.0}),
(tY:Task {uid: 12, name: "T_Y", duration: 1.0}),
(tX)-[:PRECEDES]→(tY)
```

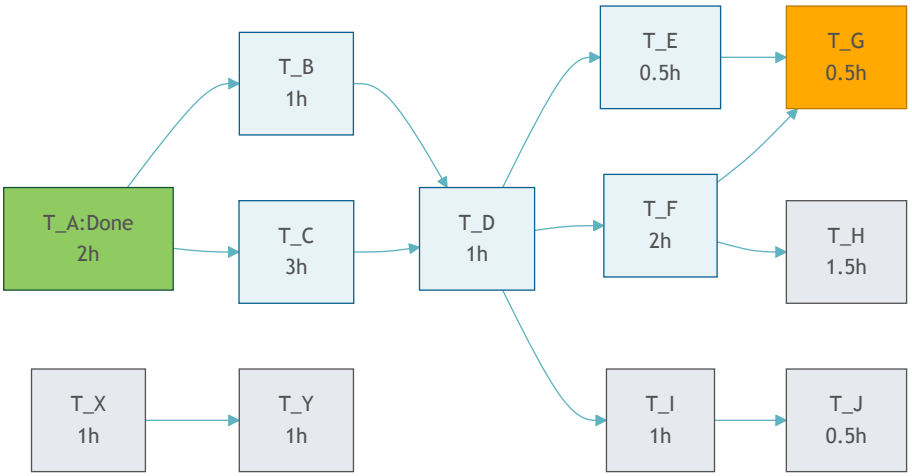

Figure 2 — Toy dataset. Green: completed (Done). Gold: scoping node (T_G). Gray: excluded by scoping.

`apoc.path.subgraphNodes` from T_G collects exactly: T_G, T_F, T_E, T_D, T_C, T_B, T_A — and excludes T_H (sibling), T_I and T_J (cousin branch), and T_X and T_Y (separate WCC). T_H and T_I/T_J are in the same WCC as T_G but are not ancestors of it — this is the case a full-graph projection would include unnecessarily.

With a naive Cypher `*` traversal from T_G, there are $2^2 = 4$ **paths** to enumerate (T_D is visited twice, once per diamond level). `apoc.path.subgraphNodes` with `NODE_GLOBAL` visits each node exactly once.

Expected CPA (skipping T_A)

T_C (3h) + T_D (1h) + T_F (2h) + T_G (0.5h) = 6.5h

3.1 Step 1 — Scope to the Ancestor Subgraph

The first query establishes T_{sub}: all ancestors of the scoping node, collected in a single graph walk. This is a standalone step here for clarity — in practice it is folded directly into the projection query in Step 3.

```
// ── Scope: collect ancestors of the scoping node ──────────
MATCH (scopingNode:Task {name: "T_G"})
CALL apoc.path.subgraphNodes(scopingNode, {
  relationshipFilter: "<PRECEDES",
  minLevel: 0
})
YIELD node AS n
RETURN n.name AS ancestor
```

> **Why `subgraphNodes` and not `spanningTree`?** A spanning tree drops edges by definition — it discovers nodes by cutting the graph into a tree structure. `subgraphNodes` collects nodes without making any claim about edges between them. Since we need all edges for the GDS projection, `subgraphNodes` is semantically correct here.

3.2 Step 2 — Node Splitting: Encoding Node Weights as Edge Weights

`gds.dag.longestPath` operates on **edge weights**. But our durations live on **nodes**. Before projecting, we need a way to encode node weights as edge weights without losing any information.

One alternative would be to design the data model around the algorithm — for instance, representing each task as a pair of `(:TimeBoundary)` nodes connected by a `[:TASK]` relationship carrying the duration. That model is native to GDS, but it is unnatural for transactional workloads: creating, updating, and querying tasks as node pairs is awkward, and the model leaks algorithmic concerns into the application layer.

The approach here makes the opposite bet: **keep the natural model** (duration as a node property, task as a single node), and perform the transformation at query time during the GDS projection. The data model stays clean for the application; the node-splitting lives entirely in the projection query, invisible to everything else.

The technique: replace each node `n` with two virtual integer IDs — an entry point and an exit point — connected by a weighted edge carrying the node's duration. Connectivity between original nodes becomes zero-weight edges between virtual IDs.

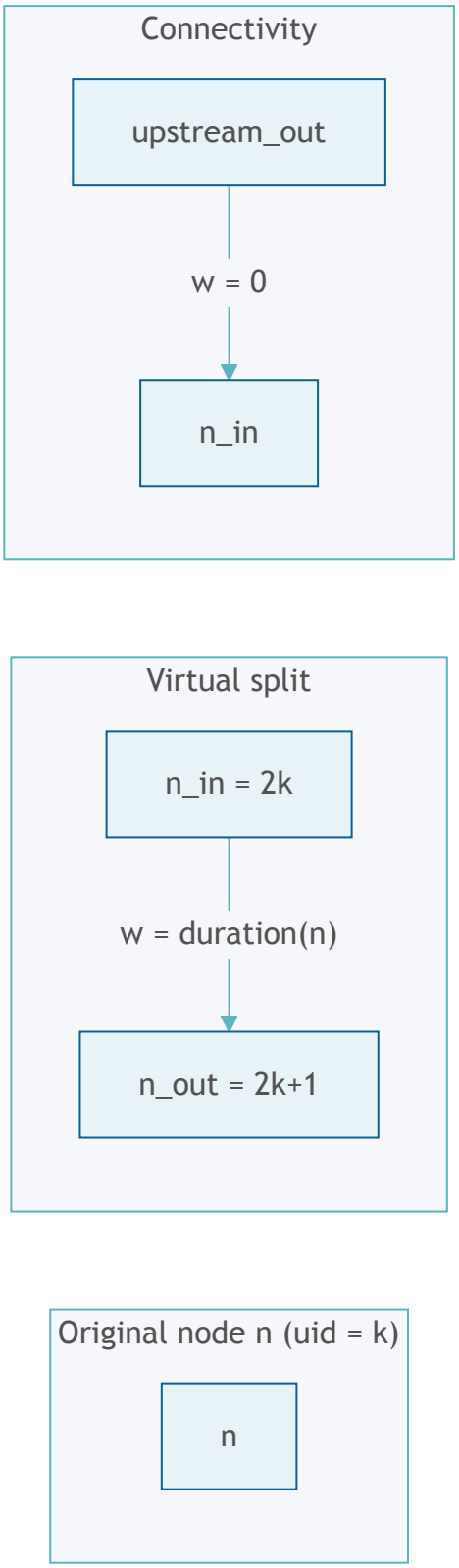

Figure 3 — Node splitting: each real node becomes two virtual IDs connected by a duration edge.

> **Why a dedicated `uid` integer property?** GDS projection expects `NODE | INTEGER` types for source and target columns. `id()` was the natural fit but is deprecated. Its replacement, `elementId()`, returns a string — unusable here. A `uid` integer property on each node is the correct modern approach.

The longest path through this virtual graph equals the sum of durations along the original node path. Mapping back is straightforward: `MATCH (n:Task {uid: virtualId / 2})`.

3.3 Step 3 — Project the Scoped Subgraph Into GDS

This is the core query. It combines the ancestor scoping from Step 1, filters out completed tasks, applies node splitting from Step 2, and projects the result into GDS — all in a single pass. Because it projects only T_{sub} rather than the full graph, it stays fast even on a large, continuously-changing graph.

```
// ── Project filtered ancestor subgraph into GDS ──────────────
MATCH (scopingNode:Task {name: "T_G"})

// 1. Collect ancestor nodes
CALL apoc.path.subgraphNodes(scopingNode, {
  relationshipFilter: "<PRECEDES",
  minLevel: 0
})
YIELD node AS n

// 2. Filter out completed tasks
FILTER NOT n:Done

// 3. Find edges — OPTIONAL MATCH ensures frontier nodes still ap
//    even when all their upstreams are Done
OPTIONAL MATCH (n)←[:PRECEDES]-(upstream:!Done)

// 4. Node splitting
WITH
  n, upstream,
  2 * n.uid              AS n_in,
  2 * n.uid + 1          AS n_out,
```

```
  2 * upstream.uid + 1   AS upstream_out,
  n.duration             AS weight

// 5. Emit both virtual edge types
CALL (*) {
  RETURN n_in  AS source, n_out AS target, weight AS w
  UNION
  WITH upstream_out WHERE upstream_out IS NOT NULL
  RETURN upstream_out AS source, n_in AS target, 0.0 AS w
}
WITH DISTINCT source, target, w

// 6. Project into GDS (reversed: scoping node becomes source)
WITH gds.graph.project(
  'cpa_graph',
  target,
  source,
  { relationshipProperties: { duration: w } }
) AS g
RETURN g.graphName AS graph, g.nodeCount AS nodes, g.relationship
```

Why the graph is projected in reverse. `gds.dag.longestPath` yields `(sourceNode, targetNode, totalCost)` where `sourceNode` is always a **source** in the GDS graph — a node with no incoming edges. In the original graph direction, the scoping node (T_G) is a sink, and the frontier tasks (T_C, T_B) are the sources. The algorithm would yield `sourceNode=T_C, targetNode=T_G` — correct cost, but no way to filter "only paths that reach my scoping node" without re-examining the full path. By projecting the graph in reverse, the scoping node becomes the source and frontier tasks become sinks. The algorithm now yields `sourceNode=T_G, targetNode=T_C` , and the `FILTER source.name = "T_G"` is direct and unambiguous. The reversal is not cosmetic — it is what makes the result filterable.

3.4 Step 4 — Run `dag.longestPath`

With the scoped, node-split graph in memory, the CPA is a single GDS call. The algorithm runs in $O(V_{\text{sub}} + E_{\text{sub}})$ using dynamic programming on the topological order — the polynomial guarantee that makes everything above worthwhile.

Two post-processing details to handle: `dag.longestPath` yields a row per reachable node (not just sinks), and `nodeIds` contains both virtual IDs for each real node. Both are resolved with the `FILTER` below.

```
// ── Compute CPA — longest path per frontier node ──────────────
CALL gds.dag.longestPath.stream('cpa_graph', {
  relationshipWeightProperty: "duration"
})
YIELD sourceNode, targetNode, totalCost, nodeIds

// Map virtual IDs back to real nodes via uid property
MATCH (source:Task {uid: sourceNode / 2})
MATCH (frontier:Task {uid: targetNode / 2})
WITH
  targetNode,
  source, frontier, totalCost,
  [id IN nodeIds WHERE id % 2 = 0 |
    head(collect { MATCH (n:Task {uid: id / 2}) RETURN n.name })
  ] AS path

// Keep only even targetNodes (n_in) to count full duration of fr
FILTER targetNode % 2 = 0
AND source.name = "T_G"
AND NOT EXISTS { (frontier)<-[:PRECEDES]-(upstream:!Done) }

RETURN
  source.name     AS scopingNode,
  frontier.name   AS criticalFrontier,
  totalCost       AS criticalPathDuration,
  path            AS criticalPath
ORDER BY totalCost DESC
```

Result

scopingNode	criticalFrontier	criticalPathDuration	criticalPath
T_G	T_C	6.5	[T_G, T_F, T_D, T_C]
T_G	T_B	4.5	[T_G, T_F, T_D, T_B]

3.5 Step 5 — Clean Up

```
CALL gds.graph.drop('cpa_graph')
```

The in-memory projection must be dropped after each query — this is not optional cleanup, it is part of the contract. Since the graph is dynamic, the next query for the same scoping node will need a fresh projection. In production, wrap Steps 3–5 in an APOC procedure so the drop is guaranteed even on failure.

§4. Why It Works: Complexity Summary

Problem	Naive approach	Cost	Fix	Cost
Collect ancestor nodes	Cypher `*` with relationship uniqueness	$O(m^d)$ — exponential	`apoc.path.subgraphNodes` with `NODE_GLOBAL`	$O(V_{\text{sub}} + E_{\text{sub}})$
Project graph into GDS	Project full graph each time	$O(m)$ — too slow at 100M edges	Project T_{sub} only	$O(V_{\text{sub}} + E_{\text{sub}})$
Find longest	—	—	`gds.dag.longestPath` on T_{sub}	$O(V_{\text{sub}} + E_{\text{sub}})$

Problem	Naive approach	Cost	Fix	Cost
path				

Node splitting doubles the virtual node count — a constant factor, not a change in complexity class.

In the production environment where this was developed, this combination brought query time from several hours on a test graph — completely infeasible in production — down to **milliseconds per scoping node**.

§5. Going Further: Real Process Monitoring Graph

The toy dataset above uses a single `Task` label and `[:PRECEDES]` for simplicity. In a real process monitoring context, the graph is richer:

- **Multiple node labels**: `Task`, `Dependency`, `SubService`, `Service`, `Metric`
- **Multiple relationship types**: `GENERATES`, `ENABLES`, `FEEDS`, `BELONGS_TO`
- **Completion state via instances**: a `TaskInstance` node with `statusCode = "COMPLETED"` linked via `IS_INSTANCE_OF` — the `:Done` label used in the toy dataset becomes a richer pattern in production
- **A `SubService` node as the scoping anchor**, carrying the SLE (Service Level Expectation)

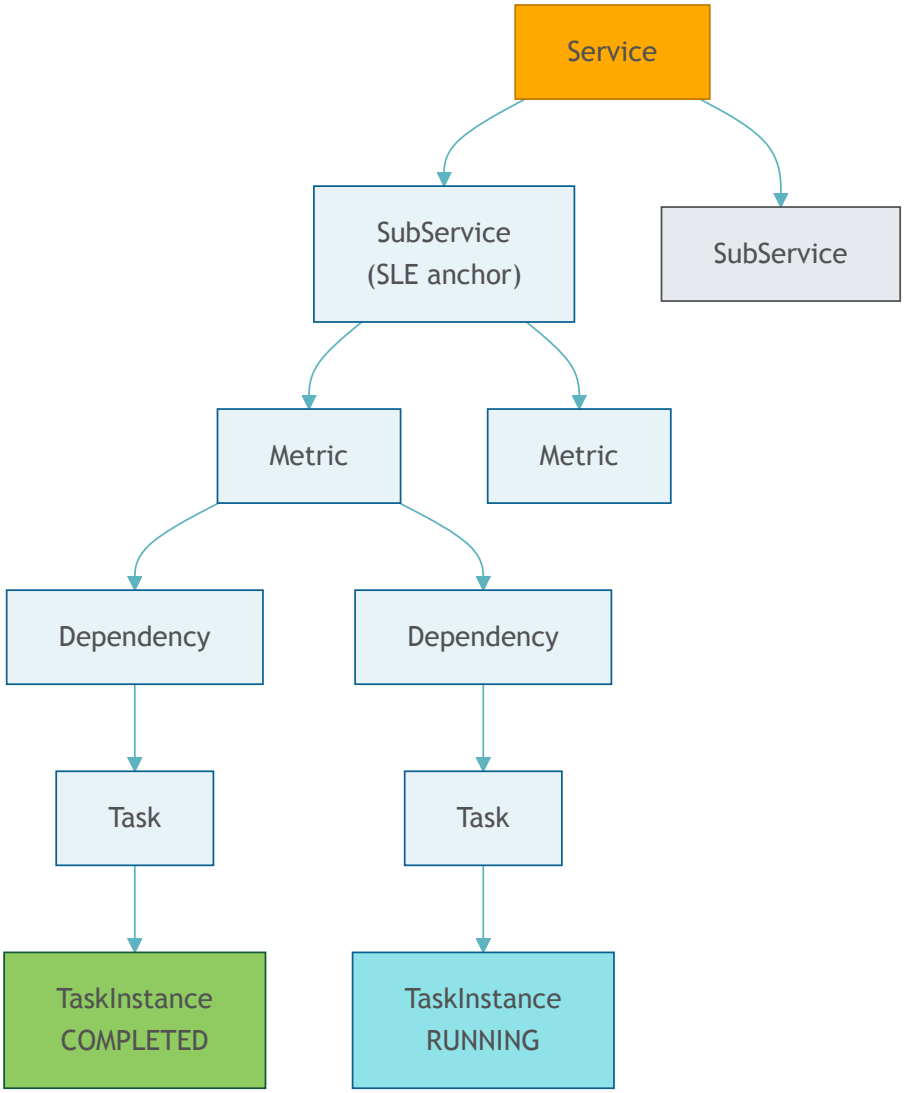

Figure 4 — Production data model: Service → SubService → Metric → Dependency → Task → TaskInstance.

The `NOT EXISTS` filter for completion becomes slightly more involved for `Dependency` nodes (which are considered done when the `Task` that generates them is completed), but the three techniques — scoping, node splitting, GDS projection — apply identically.

The full production implementation — including SLE prediction (`PREDICTED_LATE` / `ON_TRACK` / `BLOCKED`), stall detection, and a visualization layer — is available in the companion repository: halftermeyer/risk-process-monitoring-demo.

§6. Background

This approach builds on the foundations described in Unlocking DAGs in Neo4j: From Basics to Critical Path Analysis, which introduces the `gds.dag.longestPath` algorithm and demonstrates CPA on Gantt charts with physical node splitting. The present work replaces physical node splitting (creating actual `:Start` / `:End` nodes in the database) with **virtual node splitting at projection time**, keeping the data model clean and enabling real-time, scoped CPA on large dynamic graphs.